

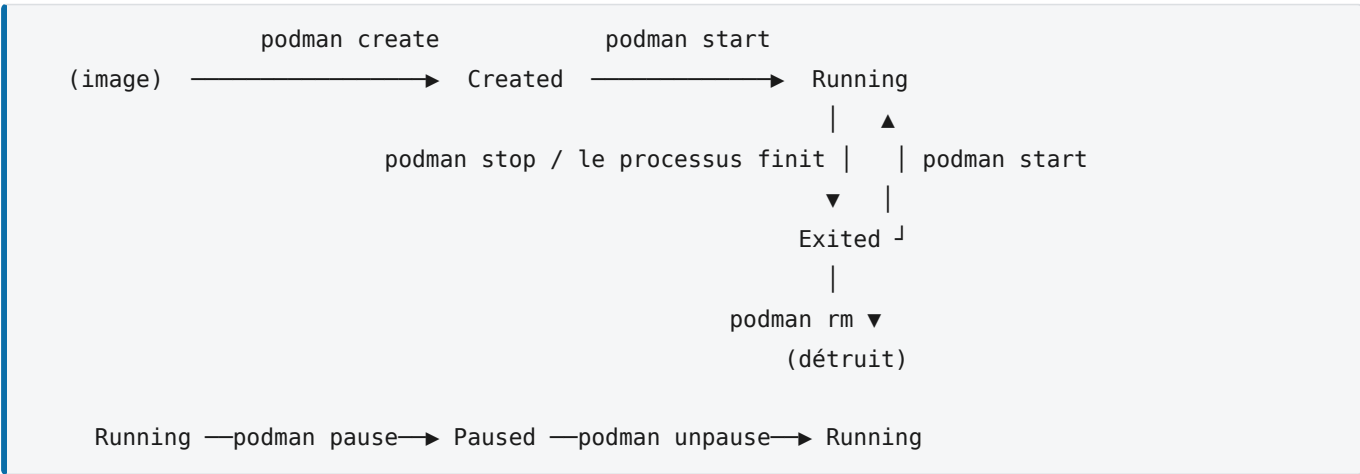
Labo 03 — Cycle de vie d'un conteneur

Théorie — naissance, vie, signaux et mort d'un conteneur ; pourquoi le PID 1 change tout, et qui redémarre vos conteneurs quand il n'y a pas de daemon.

Objectifs

- Connaître les états d'un conteneur et les commandes qui font passer de l'un à l'autre.
- Comprendre pourquoi un conteneur « s'arrête tout seul ».
- Maîtriser la relation entre le **processus principal**, les **signaux** et le **code de sortie**.
- Choisir entre `exec` et `attach`, entre premier plan et arrière-plan.
- Savoir ce que fait vraiment une *restart policy* — et ce qu'elle ne peut pas faire sans daemon.

1. Les états



Commande	Effet
<code>podman create</code>	Prépare le conteneur (couche d'écriture, config) sans le lancer
<code>podman start</code>	Démarre le processus principal
<code>podman run</code>	<code>create</code> + <code>start</code> (+ <code>pull</code> si l'image manque)
<code>podman stop</code>	Demande poliment l'arrêt, puis force après un délai
<code>podman kill</code>	Force l'arrêt immédiatement
<code>podman restart</code>	<code>stop</code> puis <code>start</code>
<code>podman pause</code>	Gèle les processus (cgroup <i>freezer</i>), sans les arrêter
<code>podman rm</code>	Détruit le conteneur et sa couche d'écriture

Un conteneur `Exited` n'est pas mort : il conserve configuration, couche d'écriture et logs. On peut l'inspecter, le redémarrer, en extraire des fichiers. Seul `rm` détruit.

2. La règle fondamentale : un conteneur vit le temps de son PID 1

À RETENIR

Un conteneur s'arrête exactement quand son **processus principal** se termine. Ni avant, ni après. Il n'y a rien d'autre à comprendre.

C'est l'explication de presque tous les « mon conteneur s'arrête tout seul » :

- `podman run alpine` sort immédiatement — la commande par défaut est `/bin/sh`, qui, sans terminal, ne lit rien et se termine.
- `podman run nginx` reste en vie — nginx tourne au premier plan et ne rend jamais la main.
- Un script qui met un service en **arrière-plan** (`java -jar ... &`) meurt aussitôt : le script se termine, donc le PID 1 aussi.

D'où une règle de conception : **une image lance son service au premier plan**. Ni démon, ni `systemd`, ni `nohup` dans un conteneur : c'est le moteur qui joue le rôle du gestionnaire de services.

→ LINUX

Sur une machine Linux normale, le PID 1 est `init` (aujourd'hui `systemd`) : le premier processus créé par le noyau, ancêtre de tous les autres. Le noyau le traite à part : s'il meurt, le système s'arrête ; et il **ignore par défaut les signaux** pour lesquels il n'a pas installé de gestionnaire, pour qu'un `kill` maladroit ne fasse pas tomber la machine. Dans un conteneur, *votre application* hérite de ce statut d'`init` — avec ses privilèges et ses pièges.

Corollaire : un conteneur est fait pour **un processus principal**. Mettre l'API et la base dans le même conteneur casse le modèle — on ne peut plus les redémarrer, les surveiller ni les mettre à l'échelle séparément.

3. Premier plan, arrière-plan, et le duo `-it`

```
podman run nginx           # premier plan : le terminal est bloqué, logs affichés
podman run -d nginx        # détaché : rend la main, affiche l'ID du conteneur
podman run -it alpine sh    # interactif : on obtient un shell utilisable
podman run --rm alpine date # exécution ponctuelle, conteneur supprimé à la sortie
```

`-it` est mal compris parce que ce sont **deux options distinctes** : `-i` garde l'entrée standard ouverte — sans elle, un shell voit son `stdin` fermé et se termine aussitôt ; `-t` alloue un pseudo-terminal — le prompt, l'écho des touches, `Ctrl+C`. En script ou en CI, `-i` seul ; en usage humain, `-it`.

PIÈGE

En premier plan, `Ctrl+C` envoie `SIGINT` au processus du conteneur : nginx s'arrête. En mode `-it`, la séquence `Ctrl+P` puis `Ctrl+Q` permet au contraire de se **détacher sans arrêter** le conteneur.

PODMAN

Avec Docker, « détaché » signifie que le daemon garde le conteneur. Avec Podman il n'y a pas de daemon : quand `podman run -d` rend la main, c'est `common` qui reste — quelques centaines de Ko, un par conteneur — pour garder les tubes `stdout / stderr` ouverts, écrire les logs et noter le code de sortie quand le PID 1 meurt. Si vous fermez votre session WSL, `common` et vos conteneurs meurent avec elle... sauf si `systemd` les tient (section 6).

4. Signaux : comment un conteneur meurt

`podman stop` n'est pas un interrupteur. Il déroule un protocole :

1. Envoi de **SIGTERM** au PID 1 : « termine-toi proprement ».
2. Attente d'un **délai de grâce**, 10 secondes par défaut (`-t` pour le changer).
3. Si le processus est toujours là, envoi de **SIGKILL**, non interceptable, immédiat — Podman l'annonce : `StopSignal SIGTERM failed to stop container ... in 10 seconds, resorting to SIGKILL`.

`podman kill` saute directement à l'étape 3. Et `podman rm -f` fait un `stop` complet, 10 secondes comprises — d'où le `-t 0` du labo 01.

→ LINUX

Un **signal** est une notification asynchrone du noyau à un processus : **SIGTERM** (15) demande l'arrêt et peut être intercepté, **SIGKILL** (9) tue sans appel, **SIGINT** (2) est le `Ctrl+C`. Un programme « gère » un signal en installant un *handler* ; sinon l'action par défaut s'applique — pour **SIGTERM**, mourir. Sauf pour le PID 1, qui n'a pas d'action par défaut : il ignore.

Pendant ces 10 secondes, une application Spring Boot bien écrite termine les requêtes en cours, ferme le pool PostgreSQL, se désinscrit du service de découverte. Avec **SIGKILL**, rien de tout cela : requêtes coupées, connexions pendantes côté base, données éventuellement incohérentes.

→ JAVA / SPRING BOOT

La JVM traduit **SIGTERM** en exécution des **shutdown hooks** (`Runtime.addShutdownHook`). Spring Boot en enregistre un qui ferme le contexte : `@PreDestroy`, pool JDBC, serveur web. Avec `server.shutdown=graceful`, le serveur cesse d'accepter des connexions et laisse finir les requêtes en cours. Tout cela **suppose que SIGTERM arrive** à la JVM.

Deux pièges qui empêchent la réception du signal :

1. Un shell intercalé devant l'application. C'est le cas d'un script de démarrage qui lance l'application sans `exec`, ou de la forme *shell* d'un `CMD` (`CMD java -jar app.jar` devient `/bin/sh -c "java -jar app.jar"`). Le PID 1 est alors `sh`, qui **ne transmet pas SIGTERM** à son enfant : Java ne reçoit jamais le signal, attend 10 secondes, puis est tué. La parade est double : forme `exec` (`CMD ["java", "-jar", "app.jar"]`) et, dans un script, `exec java -jar app.jar` en dernière ligne. Ce détail de syntaxe, détaillé au labo 04, décide de la qualité de vos arrêts.

2. Le statut particulier du PID 1. Un processus qui ne gère pas **SIGTERM** et qui tourne en PID 1 est **insensible** à `podman stop`, puis tué au bout du délai. Le PID 1 doit aussi « adopter » les orphelins, sinon les *zombies* s'accumulent. D'où `--init`, qui insère un mini-init correct (`podman-init`) devant votre application.

5. Codes de sortie

```
podman run --rm alpine sh -c 'exit 3'; echo $?      # 3
podman ps -a --format 'table {{.Names}}\t{{.Status}}'
```

Le code de sortie du conteneur est celui de son PID 1, conservé dans son statut (`Exited (3)`). Quelques codes à reconnaître :

Code	Signification usuelle
0	Terminaison normale
1	Erreur applicative générique
125	Le moteur lui-même a échoué (option invalide)
126	Commande trouvée mais non exécutable (ou <code>pasta</code> n'a pas pu ouvrir le port)
127	Commande introuvable dans l'image
137	Tué par <code>SIGKILL</code> (128+9) — <code>podman kill</code> , fin du délai de grâce, ou l' OOM killer
143	Terminé par <code>SIGTERM</code> (128+15) — un <code>stop</code> propre

137 est le code que vous verrez le plus en production : `stop` au-delà du délai de grâce, ou dépassement de la limite mémoire. `podman inspect` tranche : `.State.OOMKilled` vaut `true` dans le second cas.

6. Les *restart policies* — et qui les applique

```
podman run -d --restart=unless-stopped --name api mon-api:1.0
```

Politique	Comportement
<code>no</code> (défaut)	Aucun redémarrage automatique
<code>on-failure[:N]</code>	Redémarre si le code de sortie est $\neq 0$, au plus N fois
<code>always</code>	Redémarre toujours, y compris après un reboot de l'hôte... s'il y a quelqu'un pour le faire
<code>unless-stopped</code>	Comme <code>always</code> , sauf si vous l'avez arrêté manuellement

Chez Docker, le daemon applique ces règles, y compris au démarrage de la machine. Chez Podman, `common` relance le conteneur tant que votre session vit — mais après un reboot, personne n'est là pour lire la politique.

PODMAN

La réponse de Podman est **systemd**, le gestionnaire de services de Linux, via **Quadlet** : un fichier `~/.config/containers/systemd/api.container` de dix lignes (`[Container]`, `Image=`, `PublishPort=` ...) et `systemctl --user start api`. Le conteneur devient un service ordinaire : démarrage au boot, redémarrage sur échec, logs dans `journalctl`. C'est pourquoi Podman n'a pas voulu de daemon : celui de Linux existe déjà. Mise en œuvre au labo 10.

PIÈGE

`always` redémarre un conteneur même après un `stop` manuel, au prochain démarrage du moteur. `unless-stopped` mémorise votre intention : c'est presque toujours le bon choix sur une machine unique.

7. Observer et intervenir

```
podman logs -f --tail 50 api      # flux de sortie du PID 1
podman exec -it api sh            # nouveau processus DANS le conteneur
podman attach api                 # se rebrancher sur le PID 1 existant
podman top api                    # processus du conteneur, vus de l'hôte
podman stats api                  # consommation CPU/mémoire en temps réel
podman inspect api                # état complet, JSON
podman cp api:/app/log.txt .      # extraire un fichier, même conteneur arrêté
podman events --since 10m         # le journal des créations, arrêts, morts
```

`exec` **crée un nouveau processus** dans les namespaces du conteneur — c'est ce que vous voulez pour aller voir ce qui s'y passe. `attach` vous rebranche sur l'entrée/sortie du **PID 1 existant** : un `Ctrl+C` y arrête le conteneur.

`podman logs` ne montre que ce que le PID 1 a écrit sur `stdout` / `stderr`, capté par `common`. Une application qui écrit dans un fichier n'apparaîtra pas — d'où la règle : **logger sur la sortie standard**. C'est le défaut de Spring Boot ; ne configurez donc pas de `logging.file.name`.

8. En entreprise

- Le back Spring Boot tourne en `-d`, avec `--restart=unless-stopped` sous Docker ou comme service Quadlet sous Podman — ou sans politique sous un orchestrateur, qui gère lui-même les redémarrages.
- L'arrêt propre est un sujet de production : `SIGTERM` reçu + *graceful shutdown* Spring = déploiements sans requête perdue.
- Le diagnostic suit toujours le même enchaînement : `ps -a` (statut, code), `logs` (qu'a dit l'application), `inspect` (OOM ? configuration ?), puis `exec` si le conteneur vit encore.

À retenir

- Un conteneur vit exactement le temps de son processus principal (PID 1).
- Il faut lancer les services **au premier plan** : ni démon, ni `&`, ni `systemd` dans le conteneur.
- `-i` garde `stdin` ouvert, `-t` alloue un terminal. `stop` = `SIGTERM`, délai de grâce, puis `SIGKILL` (Podman prévient) ; `kill` = `SIGKILL` direct ; `rm -f` = `stop` complet sans `-t 0`.
- La forme `exec (["java", "-jar", "x.jar"])` est indispensable pour recevoir les signaux.
- `137` = tué (KILL ou OOM), `143` = arrêté proprement (TERM), `127` = commande introuvable.
- `rm` détruit les données du conteneur ; `stop` non. `exec` crée un processus, `attach` se branche sur le PID 1. Sans daemon, le redémarrage au boot passe par `systemd` (Quadlet).

Vocabulaire

PID 1 : processus principal du conteneur. — **délai de grâce** : temps entre `SIGTERM` et `SIGKILL`. — **graceful shutdown** : arrêt propre qui termine le travail en cours. — **restart policy** : règle de redémarrage automatique. — **OOM killer** : le noyau tue un processus quand la mémoire manque. — **zombie** : processus terminé dont personne n'a lu le code de sortie. — **common** : superviseur d'un conteneur Podman. — **Quadlet** : intégration de Podman à `systemd` (fichiers `.container`).